Machine Learning (2026) 115:247
https://doi.org/10.1007/s10994-026-07183-y

Addressing Methodological Challenges in Time Series Anomaly Detection

Nancy Xu[1] · **Henrik Boström**[1]

Received: 19 February 2026 / Revised: 12 September 2026 / Accepted: 28 September 2026
© The Author(s) 2026

Abstract
Time series anomaly detection focuses on detecting unexpected events in time series data. Traditional techniques for this task include SVMs, *k*-Nearest Neighbors and PCA, while more recent approaches frequently employ deep learning, e.g., using variational autoencoders and graph neural networks. In several studies, these more elaborate techniques are regularly found to outperform traditional approaches. We present an in-depth review of 18 influential papers on time series anomaly detection using deep learning and found recurring methodological issues which lead to questionable or inconclusive results; leakage, modified datasets, and unclear baselines. We illustrate the importance of these issues with a novel empirical investigation and demonstrate it is not trivial to replicate reported relative performance of classical and deep learning methods. We find that older methods are frequently shown to perform worse than our non optimized implementations. We fail to reject the assumption of similar performance between these older and state of the art deep learning approaches on standard benchmark datasets.

Keywords Time series · Anomaly detection · Reproducibility · Methodology

1 Introduction

Time series anomaly detection refers to the task of identifying time points within temporally ordered data where these points deviate from expected behavior. This topic is applicable to a wide range of fields, and anomalies can represent unexpected events, such as cyber attacks (Mathur & Tippenhauer, 2016), system faults (Hundman et al., 2018), or heart palpitations (Wu & Keogh, 2022). Early and accurate detection of such events is critical, and both prac-

Editors: Gianvito Pio, Jurica Levatić, Nikola Simidjievski.

✉ Nancy Xu
nancyx@kth.se

Henrik Boström
bostromh@kth.se

[1] Department of Computing and Learning System, KTH, Stockholm, Sweden

Published online: 13 October 2026

titioners and researchers need solid methodology to inform and choose the most suitable models for their use case.

With the advancements in data availability and computation power, deep learning methods have, as in many other fields, seen increased usage for time series anomaly detection. It is frequently concluded that such models perform significantly better than traditional approaches, something which is attributed to the representational power of the former (Zamanzadeh Darban et al., 2024; Garg et al., 2022).

However, as recently highlighted in several studies, deep learning papers often suffer from methodological issues. These include data leakage (Kapoor & Narayanan, 2023) and a general lack of empirical rigor (Sculley et al., 2018), which may lead to reported results being difficult or even impossible to replicate. This not only makes the verification of drawn conclusions difficult, but could also affect the trust in the field as a whole.

Recent work has begun addressing reproducibility and methodological issues within the time series anomaly detection field (Wu & Keogh, 2022; Garg et al., 2022; Liu et al., 2024; Wagner et al., 2023; Audibert et al., 2022). It has been noted that assumptions about the superiority of deep learning models may not be founded (Audibert et al., 2022; Schmidl et al., 2022). Studies have examined possible causes through the lens of insufficient benchmark datasets (Wu & Keogh, 2022; Liu et al., 2024; Wagner et al., 2023) and metrics (Garg et al., 2022; Wagner et al., 2023; Boniol et al., 2025) and have lead to the introduction of higher quality curated datasets and more informative metrics. While these are important areas for improvement, we wish to call attention to the presence of deeper methodological issues which reproducibility irrespective of metrics or datasets.

In this work, we will present an examination of 18 influential papers on time series anomaly detection using deep learning. We have through this examination also identified three main categories of methodological issues leading to unreproducibility of works in time series anomaly detection:

Leakage. Data leakage in machine learning applications has been widely discussed (Kapoor & Narayanan, 2023). Leakage refers to the problem of allowing information from the test dataset to affect model selection and evaluation procedures, for example by tuning model hyperparameters for the best test set performance. We have identified several sources of data leakage unique to time series anomaly detection.

Modifications of benchmark datasets. During our investigation, we noticed multiple instances of significant modifications being made to what are considered standard benchmark datasets. These modifications were not discussed and would be undetectable without close examination of the code base.

Underspecified experimental setup. Hyperparameters and model architectures are major factors that affect a model's performance. However, we find that such details are rarely reported in the literature or included in the code base. This makes it impossible to accurately reproduce the experiments reported in a paper.

The main question we seek to answer in this work is to what extent the identified methodological issues have affected conclusions drawn in individual studies and general recommendations regarding the effectiveness of various classes of methods.

To answer this question, we conduct a reproducibility study using using simple and state-of-the-art methods which are commonly used as baseline comparisons for time series anomaly detection. The goal is not to identify a best performing method, but further examine the impact of the above issues in reproducibility.

In Sect. 2 we summarize previous work in the time series anomaly detection field. In Sect. 3, we examine the employed methodology in the 18 selected highly influential papers and discuss identified methodological issues. We then describe the experimental setup of a novel large-scale empirical investigation in Sect. 4 and present the results in Sect. 5. Finally, we summarize our main conclusions and outline directions for future research in Sect. 6.

2 Time Series Anomaly Detection

2.1 Task Description

A time series is a sequence of data points ordered over discrete units of time, or time steps (Hamilton, 1994). A univariate time series contains one dimensional data points. A univariate time series X with m time steps is represented as:

$$X = (x_1, x_2, x_3, \dots, x_m) \tag{1}$$

where $x_i \in \mathbb{R}$ represents a data point at timestep $i \in T$ where $T = 1, 2, 3, \dots, m$.

A time series where a vector of data points is recorded at each timestep is a multivariate time series. A multivariate time series X with m time steps and d dimensions is then represented as:

$$X = (X_1, X_2, X_3, \dots, X_m) \tag{2}$$

where $X_i = (x_i^1, x_i^2, x_i^3, \dots, x_i^d)$ represents a vector at times step $i \in T$ where $T = 1, 2, 3, \dots, m$.

Due to the rare nature of anomalies, there is often a lack of sufficient labeled data for supervised anomaly detection approaches. Instead, many methods attempt to identify anomalies in an unsupervised manner by learning the representations of normal data (Zamanzadeh Darban et al., 2024; Garg et al., 2022). In unsupervised anomaly detection the question becomes: How to detect that an observation (a single value or vector) at time step t, or within a set of observations in a window of time steps $t, \dots, t + w$, is anomalous with respect to previous observations ($(x_1, x_2, x_3, \dots, x_t)$ or $(X_1, X_2, X_3, \dots, X_t)$, respectively)?

The standard experimental setup for unsupervised time series anomaly detection using deep learning methods is as follows (Zamanzadeh Darban et al., 2024; Liu et al., 2024). The setup starts with a pair of time series; the train set and its corresponding test set. This pair is commonly referred to as an entity in the wider literature and may be univariate or multivariate (Garg et al., 2022). The train set is unlabeled, as it is assumed to contain no anomalies. In contrast, the test set includes binary labels indicating the presence of anomalies at certain time steps. In this experimental setup, the assumption is that the test set follows the training set in time, but not necessarily directly after. Additionally, it is common to withhold the last segment of the train set as a validation set to measuring progress during training.

The input to a deep learning algorithm is a subset of the a time series called a window. Each window is a sequential subset of a predefined length, and windows are extracted by iterating through the time series. The set of windows W of length s extracted from a time series is then represented as:

$$W = (w_1, w_2, \ldots, w_{m-s}) \tag{3}$$

where $w_i = (x_i, \ldots, x_{i+s})$.

A model is trained on windows extracted from the train set and then evaluated on windows from the test set. During testing, the model's predictions are used to generate anomaly scores for the window. These scores can be assigned on a per-window basis where the entire window is given a single score, or a per-timestep basis where each timestep in the window is scored. These anomaly scores are used to determine whether an anomaly is present.

2.2 Approaches

2.2.1 Classical

Time series anomaly detection is a long standing area of research. Classical techniques use a variety of approaches to classify anomalies.

Support Vector Machines (SVMs) (Ma & Perkins, 2003) learn a hyperplane boundary around normal observations, and observations outside of these boundaries are considered anomalous. *k*-Nearest Neighbors (*k*-NN) (Ramaswamy et al., 2000; Chaovalitwongse et al., 2007) use the distance of an observation from it's nearest neighbors as the anomaly score.

Principal Component Analysis (PCA) has also been used for anomaly detection using the subspace method (Lakhina et al., 2004). Observations are decomposed into their constituent eigenvectors and then reconstructed, the difference between the reconstruction and original is used as the anomaly score. In addition to these older algorithms, new approaches in this family continue to be developed (Yeh et al., 2016; Taylor & Letham, 2018; Benkabou et al., 2025).

2.2.2 Deep Learning

Deep learning methods for anomaly detection have rapidly gained popularity for their representational power and detection performance (Zamanzadeh Darban et al., 2024; Garg et al., 2022). Below, we categorize the deep learning time series anomaly detection methods by the strategy used to obtain anomaly scores, i.e., forecasting and reconstruction, following (Zamanzadeh Darban et al., 2024).

Forecasting methods are trained to predict the values of a timestep or sequence of timesteps after the input window. The model's prediction is compared to the true observation at that timestep, and the difference is used as the anomaly score. This format allows model families that are commonly used for time series forecasting tasks to also be used for anomaly detection, such as LSTMs (Malhotra et al., 2015), GRUs (Nanduri & Sherry, 2016), CNNs (He & Zhao, 2019; Wu et al., 2023), and Transformers (Tuli et al., 2022). Recent Graph Neural Networks (GNNs) methods like GTA (Chen et al., 2021) and GDN (Deng & Hooi, 2021) are forecasting GNN methods capable of learning graph edges, allowing them to be applied to more general benchmark datasets that do not include graph structured data.

Reconstruction methods learn the underlying distribution by reconstructing the input window. The reconstruction is compared to the input, and the difference is used to determine the anomaly score. Basic autoencoders (AEs) (Sakurada & Yairi, 2014) use feed forward

Table 1 Summary of methdological problems found during the investigation

Method Name	Year	Citations	No Repo	Metrics	Leakage		Dataset	Underspecification	
					Thresh.	Point-Adjust	Modified	In Paper	In Repo
Autoencoder (Sakurada & Yairi, 2014)	2014	1897	x	AUC-ROC	-	-	-	o	-
LSTM-AD (Malhotra et al., 2015)	2015	2354	x	P, R, F0.1	o	-	-	o	-
DAGMM (Zong et al., 2018)	2018	2846	x	P, R, F1	x_2	-	-	o	-
LSTM (Hundman et al., 2018)	2018	2209	o	P, R, F0.5	o	-	o	-	-
LSTM-VAE (Park et al., 2018)	2018	1297	x	AUC-ROC	-	-	-	x	-
DeepAnt (Munir et al., 2019)	2018	929	x	P, R, F1, AUC-ROC	o	-	o	x	-
Omnianomaly (Su et al., 2019)	2019	2071	o	P, R, F1	x_1	x	x	x	x
TCN (He & Zhao, 2019)	2019	320	x	P R F1	x_1	o	-	-	-
MADGAN (Li et al., 2019)	2019	1643	o*	P, R, F1	?	o	-	x	x
BeatGAN (Zhou et al., 2019)	2019	451	o	AUC-ROC, AP	-	o	x	x	x
USAD (Audibert et al., 2020)	2020	1411	o*	P, R, F1	x_1	*	x	x	x
RANSynCoders (Abdulaal et al., 2021)	2021	502	o	P, R, F1	o	x_3	?	x	x
Interfusion (Li et al., 2021)	2021	438	o	F1	x_1	x	o	x	x
GDN (Deng & Hooi, 2021)	2021	1609	o	P, R, F1	x_1*	o	-	x	x
Anomaly Transformer (Xu et al., 2022)	2021	1241	o*	P,R, F1	x_2*	x	x	x	x
GTA (Chen et al., 2021)	2022	632	o*	P, R, F1	x_1	x	x	x	x

Table 1 (continued)

Method Name	Year	Citations	No Repo	Metrics	Leakage		Dataset	Underspecification	
					Thresh.	Point-Adjust	Modified	In Paper	In Repo
TranAD (Tuli et al., 2022)	2022	1136	o	P, R, F1, AUC-ROC	x_1	x	x_4*	x	o
TimesNet (Wu et al., 2023)	2023	3031	o	F1	x_2	o	x	x	o

'x' marks the presence of the problem, 'o' marks absences, and '-' marks when the issue is not applicable. A '*' modifier indicates discrepancy between the paper and implementation. '?' is used when information related to these issues is missing. 'x_1' and 'x_2' indicate thresholding for the best score and by data distribution respectively. 'x_3' and 'x_4' indicate unique variants of the point-adjust algorithm and dataset usage

networks to directly encode and decode sequences. Later works have seen AEs incorporated into Gaussian Mixture Modes (Zong et al., 2018) and ensemble models (Abdulaal et al., 2021). Variational auto encoders (VAEs) (An & Cho, 2015) instead compresses the input into the parameters of a distribution within the latent space and the decoder samples from the latent space to reconstruct the input. This approach has generated several variations, such as replacing the linear layers with different sequential layers, for example LSTMs in LSTM-VAE (Park et al., 2018) and GRUs in Omnianomaly (Su et al., 2019), or using hierarchical approaches (Li et al., 2021). Some methods incorporate adversarial approaches for reconstructing the input sequence where a discriminator is trained to distinguish between the original and reconstructed input window and the generator is trained to fool the discriminator. Some methods such as USAD (Audibert et al., 2020) and BeatGAN (Zhou et al., 2019) only use the reconstruction loss from the trained generator for the anomaly score, while others like MADGAN (Li et al., 2019) incorporate the discriminator as well.

3 Examination of Previous Work

3.1 Inclusion Criteria

Our examination is based on 18 highly influential papers in the time series anomaly detection area. Papers were selected by examining several time series anomaly detection surveys (Wu & Keogh, 2022; Zamanzadeh Darban et al., 2024; Garg et al., 2022). We then filter for papers published after 2014 with more than 300 citations. When possible, we also inspect the official repositories associated with these papers. A summary of the results of the investigation is given in Table 1. We list each paper by the name of the proposed method, along with the publication date and number of citations recorded by Google Scholar as of 12/02/2026.

An 'x' is used to indicate the presence of the problem described by the column. Conversely, an 'o' indicates that the problem is no present in the examined work. For example, an 'x' in the *No Repo* column indicates the lack of an official repository. When there is conflict between what is written in the paper and what is implemented in the public repository, we indicate this by reporting what is seen in the repository and add an asterisk '*'. For

example, an 'o*' in the *No Repo* column indicates that no repository was listed in the paper, but an official implementation can be found online.

The *Metrics* column records the metrics used in the paper to evaluate anomaly detection performance.

The following columns record the presence of methodological errors which have been grouped into three categories; leakage, dataset, and underspecification. These columns are further discussed in the following sections.

3.2 Leakage

Since anomaly scores are continuous values, the scores must be thresholded to produce binary predictions in order use the classification metrics of precision, recall, and F1 score. In the column named *Thresholding* in Table 1, two categories of problematic threshold selection procedures are indicated.

The most common of these categories (labeled x_1 in Table 1), includes approaches that select a threshold by evaluating a set of thresholds on the test dataset and choosing the threshold that results in the highest performance. While this method results in high reported scores, in practice this gives an unrealistic expectation of performance based on information that does not exist in practice at the time the prediction is made. This approach is used in seven out of the 18 examined papers.

The next category of dubious thresholding (labeled x_2), is another post-hoc method where the threshold is set to a value that makes a percentage of the all the test predictions anomalous. The percentage is tuned per dataset, often based on the known proportion of anomalies in the test set, e.g., if it is known that 5% of the test set is anomalous, the threshold is set so 5% of predictions are anomalous. However, the method TimesNet (Wu et al., 2023) appears to use an unknown criteria to determine the percentage used for each dataset. While this is more fair than optimizing directly for the best score, it is still basing the choice of this parameter on test dataset knowledge. Three of the papers use this method.

The rest of the papers either use evaluation metrics that do not require thresholding, indicated by a ('-') in the column *Thresholding*, or thresholding methods that do not rely on test set information, indicated by ('o'). In the case of the MADGAN (Li et al., 2019) method, we are unable to determine the thresholding method used, which we mark with ('?'). There is no description of thresholding in the paper and the repository uses a fixed value, although anomaly detection performance reported by users on their repository do not resemble the performance presented in the paper.

Another common area of leakage is the use of the point-adjust algorithm during evaluation. The point-adjust algorithm is a post-hoc modification to thresholded predictions before scoring. The prediction is compared to the ground truth labels, if at least one anomalous point is correctly detected in a continuous segment of anomalies, the entire segment is then considered to be correctly detected. This adjustment strategy is illustrated in Fig. 1.

The motivation behind this technique is the assumption that as long as any part of an anomalous event is detected, it is acceptable to consider the entire event correctly detected. However, adjusting predictions to match ground truth labels is a clear case of leakage from the test set. Additionally, this adjustment allows random guessing to perform competitively with state of the art methods, as shown in (Siwon et al., 2022). Of the papers we have examined, six papers are directly using this method (indicated by an x in the *Point-Adjust*

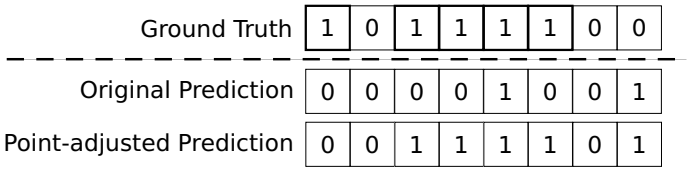


Fig. 1 Illustration of the point-adjust algorithm. The top row shows the ground truth anomaly labels. The second row is the original anomaly predictions after thresholding. The last row shows the new predictions after point-adjust has been applied. The original predictions would score 0.5 precision, 0.2 recall, and 0.286 F1. The point adjusted predictions would score 0.8 precision, 0.8 recall, and 0.8 F1

column). The RanSynCoders method uses a modified version of point-adjust (labeled x_3). Instead of affecting the entire anomaly sequence, only the false negatives directly after a true positive are modified. While this affects the precision, recall, and F1 metrics to a lesser degree than point-adjust, information from the test set is still being used for evaluation. USAD claims to use point-adjustment but in practice seems to use the term to refer to evaluating predictions on a per window level instead of per time step. Since it is a reformulation of the anomaly detection task from per time step to per window, this is not a case of leakage. Methods which were published before the point-adjust algorithm was introduced are marked with a ('-').

3.3 Modified Datasets

Anomaly detection datasets may contain multiple entities which should be considered separately. These entities may not align temporally, be the same length, or even be from the same source. Therefore, no assumptions can typically be made about relationships between entities.

The most commonly occurring multi-entity datasets in our selection of papers are the Soil Moisture Active Passive (SMaP) and Mars Science Laboratory (MSL) datasets, where each entity represents recordings from a different spacecraft sensor, and the Server Machine Dataset (SMD) where each entity represents recordings from different servers (Hundman et al., 2018; Su et al., 2019).

On all examined papers using such datasets, the performance on multi-entity datasets is reported with a singular score. Only one paper states that the reported score is an average of model performance across all entities (Li et al., 2021). While it may appear reasonable to assume the average is being used in all such cases, this assumption allows major inconsistencies in dataset preprocessing to pass unnoticed. When we investigated how these scores were aggregated, we discovered that frequently these scores were not aggregates at all. Instead, multi-entity datasets have been modified to quite some extent and are being evaluated as if they are a single entity.

The most common modification of these datasets occurs in the form of concatenating entities together. This is typically done by combining entities into a single longer time series, although modifications where SMaP/MSL sensors have somehow been temporally aligned and then concatenated along dimensions has also been observed. Concatenating entities introduces a temporal order that does not exist in reality, violating the basic assumption of temporally ordered data for time series anomaly detection. Ten papers in our examination include multi-entity datasets in their experiments, and six have been found to have

concatenated at least one in their official repositories. These modifications are not discussed in the papers and in some cases, are even being distributed in public repositories without disclosure.

In (Tuli et al., 2022), TranAD is evaluated on a subset of entities from each dataset (indicated by x_4 in the *Modified Dataset* column). While the use of subsets is clearly described in the paper, the instructions on their repository for how to reproduce the results only concerns a single entity per dataset rather than a subset.

Methods which do not include multi-entity datasets in their experiments, use non public datasets, or where the properties of the datasets are ambiguous, are marked with a ('-') in the *Modified Dataset* column. The authors of RanSynCoders reports scores on multi-entity datasets, but the official repository contains no insights on how datasets were loaded or evaluated (Abdulaal et al., 2021). This has been marked with ('?').

3.4 Underspecified Experimental Setup

Comparing the performance of a proposed model against previous approaches, referred to as baseline methods, is the standard method of reporting results in machine learning. In some research areas it is common to use scores reported by previous papers instead of rerunning each baseline method itself, but in time series anomaly detection there is a wide variety of window sizes being used across papers. In our investigations, we have seen window sizes ranging from three to three hundred. This makes directly comparing against previously reported scores difficult as the window size parameter directly affects the number of sliding windows that can be extracted and used from the test dataset for evaluation. Since datasets and window sizes differ from paper to paper, readers can conclude that authors conduct their comparisons by running the baseline methods themselves.

In both classical and deep learning methods, hyperparameters greatly affect the performance of the model. While providing implementation and hyperparameter details of proposed methods is common, the same details for baseline models used for comparison is often neglected; of the 18 papers examined, only three report baseline hyperparameters in the text, but none have official repositories. Two papers provide baselines implementations in the official repository but without disclosure in the text.

Furthermore, official implementations of highly cited baseline methods may be unavailable, nonfunctional, or simply out of date. For example, the official MADGAN repository uses Tensorflow 1 and has not been updated in eight years[1] (Li et al., 2019). While readers may assume these methods are reimplemented when used as comparison methods by later authors, we can see in public implementations that there can be variability in interpretation of the original work. These three public implementations of MADGAN[2][3][4] vary in their implementation of MADGANs invert mapping module, in one case excluding it completely. Such details may affect the method's performance, but in a scientific study it is difficult to judge the presence or severity of the divergence without documentation.

[1] https://github.com/LiDan456/MAD-GANs

[2] https://github.com/Guillem96/madgan-pytorch/

[3] https://github.com/kobeHub/madgan/

[4] https://github.com/Glebzok/MAD-GAN/

Best practices for comparing the detection performance of deep learning models and classical models are unclear. Many recent classical methods for time series anomaly detection such as MERLIN (Nakamura et al., 2020) and Matrix Profile (Yeh et al., 2016) consider the time series in it's entirety, and assume each subsequent input directly follows the previous temporally. In contrast, deep learning methods make no such assumptions between input windows. Although each window is internally ordered temporally, a deep learning model evaluates each window independently from the others.

This causes difficulties for comparison due to the train/test set paradigm widely used to train deep learning models. A deep learning model can evaluate any window extracted from a test set while a classical model may require all the preceding time steps in order to evaluate that same window. How the disjointed data is handled, for example by restricting classical models to only the test set or allowing them access to the entire train and test sets, is rarely, if ever, addressed in the literature when classical methods do appear as baselines.

We indicate which papers report hyperparameters of baseline methods in the *In Paper* column. Methods which did not compare against any baselines are marked with a ('-'). The *In Repo* column indicate the presence of baseline methods in the official repository, methods which do not have a public repository as marked with a ('-').

4 Empirical Investigation

We perform an empirical investigation to re-examine the relative performance of common anomaly detection methods used as baseline models in literature without the interfering effects of leakage and modified datasets. The aim of this investigation is not to find a best performing method, but attempt to replicate commonly reported performance measurements in the examined papers (see Table 1).

4.1 Datasets

We examine six popular benchmark datasets for time series anomaly detection. Some of the datasets contain multiple independent time series, which we refer to as entities as described in Sect. 3.3. Information about anomaly rates and types can be found in Table 2.

Secure Water Treatment (SWaT) (Mathur & Tippenhauer, 2016)/Water Distribution (WaDi) (Ahmed et al., 2017): SWaT and WaDi are single entity datasets released by the iTrust Center for Research in Cyber Security. SWaT consists of readings from a replica of a modern water treatment facility which is then attacked by researchers using cyber-physical methods. This dataset records 51 variables across 11 days of operation, 7 running normally and 4 with attacks. Similar to SWaT, WaDi is formed of sensor readings from a replica of a

Table 2 Anomaly rates and types present in each dataset

Dataset	Anomaly rate	Anomaly types
SWaT	12.2%	P, C, Col
WaDi	5.9%	P, C, Col
SMAP	13.1%	P, Col
MSL	10.7%	P, Col
SMD	4.16%	P, Col
UCR	0.51%	P, Col

P: Point Anomaly, C: Contextual anomaly, Col: Collective anomaly. Anomaly type analysis taken from Audibert et al. (2022); Liu et al. (2024)

modern water distribution facility which is then attacked. This dataset records 123 variables over 16 days of operation, 14 running normally and 2 with attacks. iTrust has stated that the SWaT system required five hours to reach it's normal operating state (Goh et al., 2017), therefore we have truncated those first five hours. We also observed previous work often employs downsampling, reducing the length of the time series by averaging over some number of timesteps (Chen et al., 2021; Deng & Hooi, 2021; Audibert et al., 2020). We will follow and downsample by a factor of 5 as in (Audibert et al., 2020).

Soil Moisture Active Passive (SMAP) and Mars Science Laboratory (MSL) (Hundman et al., 2018): These are two collections of multivariate timeseries obtained from satellite sensor readings released by NASA. SMaP contain 55 entities of 25 dimensions and MSL 27 entities of 55 dimensions. Previous work has criticized these datasets for containing both trivial and mislabeled anomalies, arguing that the frequency of the errors make them unsuitable for measuring detection performance (Wu & Keogh, 2022). Despite such criticism, these datasets are widely used and therefore have been included in this study for comparison.

Server Machine Dataset (SMD) (Su et al., 2019): This multi-entity dataset consists of 38 performance metrics of 28 servers collected over a five week period from a large internet company. It was released together with the Omnianomaly method.

The University of California Riverside Time Series Anomaly Archive (UCR) (Wu & Keogh, 2022): This dataset consists of 250 univariate entities recorded from a wide variety of subjects including ECG readings, power demand, and accelerometers. We remove entities which contain artificial perturbation as described by We et al. and which are too large to for an experiment to successfully run on an NVIDIA A40, leaving 125 entities in total. These entities are listed and further grouped by subject in Appendix 1.

4.2 Experimental Setup

All methods share the same preprocessing steps. The given train and test data splits are scaled using a standard scaler fitted to the train dataset. As is standard in deep learning, a portion of the training data is reserved for validation of the model's progress during training. We have chosen to reserve the last 10 percent of the training data for validation. To avoid possible bias against models which operate better on short or long windows, we conduct three experiments per dataset with sliding window sizes of 12, 48, and 128. Only results for window sizes that are smaller than the validation set will be included.

Forecasting and reconstruction methods take different approaches when generating predictions, we choose to focus on reconstruction based deep learning methods to minimize differences in evaluation procedure. We also choose two classical methods which work within this evaluation procedure, as outlined in Section 2.1. Code for this project is available at https://github.com/xun468/AdMeCh-TSAD.

4.3 Evaluation

As discussed in Sect. 3.2, the standard classification metrics of precision, recall, and F1 require thresholding. An alternative metric in time series anomaly detection which does not require thresholding is the Area Under the Receiver Operating Characteristic Curve (AUC-ROC) (Zamanzadeh Darban et al., 2024). The metric reflects the ranking performance and

can in our context be interpreted as measuring the probability of ranking an anomaly ahead of a non-anomaly.

While AUC-ROC is a common evaluation metric independent of thresholding choice, it has been observed to be too optimistic on datasets with large class imbalances. Due to the rarity of anomalies, anomaly detection datasets are typically also imbalanced. To examine classification performance on the rarer positive class, we also include measurements using Area Under the Precision Recall Curve (AUC-PRC), a threshold independent metric that is more informative under class imbalances in Appendix 2.

For the single entity datasets SWaT and WaDi, we take the average score over five runs using the same train/test splits.

In the case of multi-entity datasets, we conduct statistical hypothesis testing, comparing multiple methods over multiple datasets, using the Friedman test followed by the Nemenyi post-hoc test to identify which groups are significantly different from each other with $\alpha = 0.05$ (Demšar, 2006). The tests were conducted using the autorank library (Herbold, 2020).

We do not use repeated runs on multi-entity datasets. The Friedman test assumes each dataset is drawn independently from the same underlying distribution. This assumption precludes repeated use of the same dataset as duplicates are not independent samples. Previous work has also shown that the natural variation between training runs is balanced through the rankings across datasets, or in this case entities (Demšar, 2006).

While it cannot be strictly confirmed, we assume that entities with similar origins and variables is sufficient to fulfill the second assumption of a shared underlying distribution. Although the UCR dataset consists of a variety of subjects, following previous work we will evaluate the dataset as a whole. In Appendix 1 we also present results on the individual subject groups.

4.4 Methods

We include two commonly appearing classical baselines, *k*-Nearest Neighbors (*k*-NN) and Principle Component Analysis (PCA), in the investigation. These methods do not require their inputs to be temporally continuous, which makes them suitable for the standard evaluation procedure, as described above. Both methods are implemented using the scikit-learn package (Pedregosa et al., 2011). We use k=10 for the *k*-NN and the use the scikit-learn default settings for choosing number of PCA components. Both methods are fitted with the train data split without windowing (see Sect. 2.2.2) and are evaluated on windowed test data. The validation split remains withheld and is not used.

Since a *k*-NN is deterministic algorithm and generates distance scores on a point-by-point basis, there is no difference between windowing the test dataset and then evaluating every window, and evaluating on the unwindowed test dataset and then windowing the resulting distance scores. To save computation time, we perform the latter.

The PCA follows a similar evaluation procedure to deep learning models. The fitted PCA evaluates each test set window by transforming the window into it's lower dimensional representation and then inverting the transformation. The anomaly score is the mean squared error between the original sequence and the recovered sequence.

We examine popular deep learning that frequently appear in the literature as baselines, namely AE (Sakurada & Yairi, 2014), VAE (An & Cho, 2015), LSTM-VAE (Park et al.,

2018), MADGAN (Li et al., 2019), Omnianomaly (Su et al., 2019), and USAD (Audibert et al., 2020). These methods were implemented using Pytorch (Paszke et al., 2019).

Our implementation of Omnianomaly is based on the implementation[5] by Liu et al. (2024) as of 19 February 2026. We note that on 15 April 2026 the repository has been updated with a new implementation. Initial investigation showed a decrease in stability when using the original parameters specified by Su et al. (2019), therefore we continue with the older implementation. Our USAD implementation follows their official repository[6] (Audibert et al., 2020). Implementations for these and all other methods in this study can be found in our official repository.

Model hyperparameters such as the number of hidden dimensions at each layer and optimizer settings are a critical component of deep learning model performance. However, due to underspecification these critical settings for baseline experiments are rarely available.

When available, we use hyperparameters and optimizers as specified by the original authors in the papers or repositories of the baseline method for each dataset. If these hyperparameters are not available, we use fixed values for the hidden dimension dependent on dataset and the default Adam optimizer. Table 3 contains the hidden unit hyperparameter used for each baseline on each dataset. Furthermore, we intentionally use suboptimal settings for *k*-nn and PCA to maximize the ability to replicate the relative performance as seen in previous studies. The assumption is that there should be a clear difference between deep learning models with optimized parameters, deep learning models with unoptimized parameters, and classical models with unoptimized parameters.

Models are trained for 50 epochs and the parameters with the best validation reconstruction performance are selected for evaluation on the test set. Further implementation and hyperparameter details can be found in our publicly available code.

5 Results

Figures 2 and 3 show the results of the Friedman and Nemenyi tests in the form of critical difference (CD) diagrams, where CD indicates the difference in average ranks that is required for the null hypothesis to be rejected for any pair of methods. Figure 2 shows the anomaly detection performance of the models on the SMAP and MSL datasets and Fig. 3 shows the performance on the SMD and UCR datasets. The position of each method indicates it's average ranked AUC-ROC performance across all entities in the dataset, where

Table 3 Hidden dimensions used for each dataset

	SWaT	WaDi	SMAP	MSL	SMD	UCR
AE	128	128	64	64	64	64
VAE	128	128	64	64	64	64
LSTM-VAE	128	128	64	64	64	64
MADGAN	**100**	**100**	64	64	**100**	64
Omnianomaly	500	500	**500**	**500**	**500**	500
USAD	**40**	**100**	**55**	**33**	**38**	55

Bold indicates hyperparameters taken from original papers

[5] https://github.com/thedatumorg/TSB-AD

[6] https://github.com/robustml-eurecom/usad

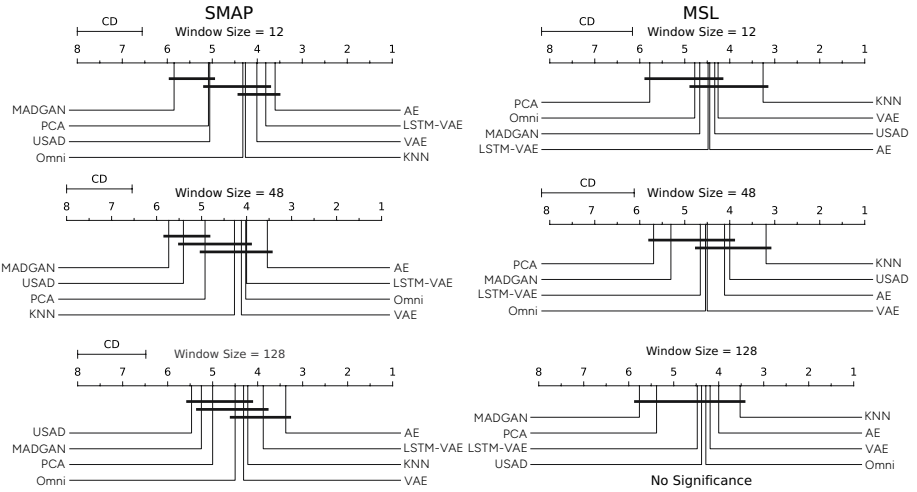


Fig. 2 Critical difference diagrams for AUC-ROC scores ($\alpha = 0.05$) on the SMAP and MSL datasets at various window lengths

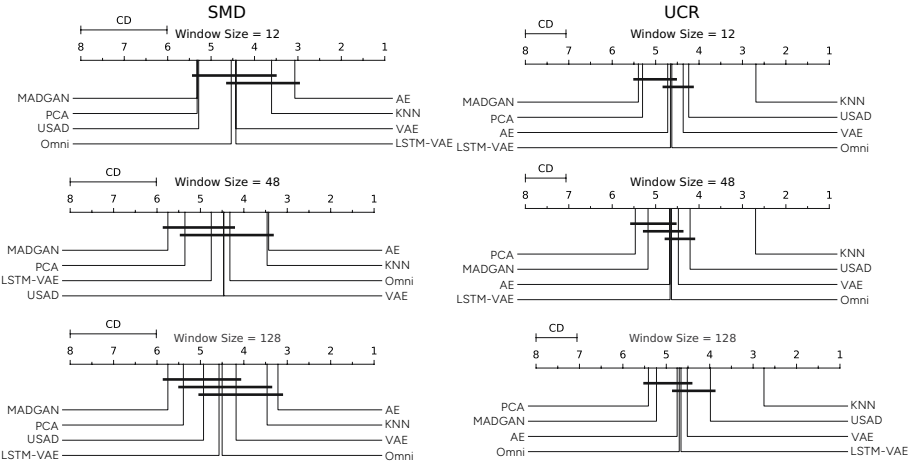


Fig. 3 Critical difference diagrams for AUC-ROC scores ($\alpha = 0.05$) on the SMD and UCR datasets at various window lengths

better performance is given a numerically lower rank (e.g., the top performing model is ranked 1). The bars indicate similarly performing groups where there is no significant difference between the performance of the members in a group. The Friedman test does not allow for rejecting the null hypothesis that there is some difference in performance between any of the methods for some combinations of dataset and window sizes. For these combinations, the post-hoc test is not employed.

We can see in the figures that the groups of similarly performing models are quite large and in many cases overlap considerably. For SMAP, MSL, and SMD datasets there is no clear best performing method or group of methods. Top groups contain a mix of both deep

Table 4 Number of appearances in the top performing group

Method	# top
k-NN	11
PCA	2
AE	8
VAE	8
LSTM-VAE	8
Omnianomaly	8
MADGAN	1
USAD	4

Non-significant results are not counted

Table 5 Comparison of mean and standard deviation of the AUC-ROC score on the SwaT dataset at window size 12, 48, and 128 across five runs

Model	SwaT-12	SwaT-48	SwaT-128
k-NN	**0.824**	0.824	0.824
PCA	0.813	0.812	0.812
AE	**0.826 ± 0.001**	**0.826 ± 0.003**	0.822 ± 0.003
VAE	0.820 ± 0.0	0.820 ± 0.0	0.820 ± 0.0
LSTM-VAE	0.820 ± 0.0	0.820 ± 0.0	0.820 ± 0.0
Omnianomaly	0.818 ± 0.006	0.656 ± 0.265	0.696 ± 0.279
MADGAN	0.816 ± 0.010	0.831 ± 0.020	0.818 ± 0.020
USAD	0.825 ± 0.024	0.813 ± 0.003	0.817 ± 0.007

Bold indicates highest score

Table 6 Comparison of mean and standard deviation of the AUC-ROC score on the WaDi dataset at window size 12, 48, and 128 across five runs

Model	WaDi-12	WaDi-48	WaDi-128
k-NN	0.518	0.518	0.517
PCA	0.495	0.495	0.494
AE	0.528 ± 0.005	0.526 ± 0.008	0.524 ± 0.005
VAE	**0.551 ± 0.0**	**0.551 ± 0.0**	**0.551 ± 0.001**
LSTM-VAE	**0.551 ± 0.0**	0.551 ± 0.001	**0.551 ± 0.0**
Omnianomaly	**0.551 ± 0.0**	**0.551 ± 0.0**	0.550 ± 0.001
MADGAN	0.456 ± 0.035	0.435 ± 0.052	0.505 ± 0.052
USAD	0.517 ± 0.007	0.505 ± 0.031	0.541 ± 0.004

Bold indicates highest score

learning and classical models, Table 4 records how many times each method appears in the top group. Surprisingly, *k*-NN often appears in the top group and is the best performer on the UCR dataset.

In the case of single entity datasets, the AUC-ROC performance on the SWaT dataset is similar across all models, with a basic autoencoder as the top performer by a small margin, as seen in Table 5. Results on the WaDi dataset (Table 6) are similar albeit with VAE, LSTM-VAE, and Omnianomlay tied as the top performers.

On all datasets, the increase in model complexity and hyperparameter tuning do not consistently lead to performance gains and we do not see sharp performance decreases even with purposefully suboptimal usage of traditional methods. These findings contradicts previously published studies which report traditional methods significantly under perform when compared to deep learning methods on anomaly detection tasks, as can be seen in baseline tables

such as Zamanzadeh Darban et al. (2024); Wu et al. (2023); Tuli et al. (2022); Xu et al. (2022); Deng and Hooi (2021); Abdulaal et al. (2021); Li et al. (2019); Munir et al. (2019).

Additional results using Area Under the Precision Recall Curve (AUC-PRC) metric can be found in Appendix 3. In general, the results using AUC-PRC resemble that of AUC-ROC. Groups of similarly performing models on multi-entity datasets are large and overlapping. Tuned deep learning methods do not perform significantly better and unoptimized traditional methods do not perform significantly worse. Notably, PCA joins the top performing group with *k*-nn on the UCR dataset using AUC-PRC.

We present a further investigation on the possible effects of hyperparameter choice for untuned models is further explored in Appendix 2 and a look at VAE, LSTM-VAE, and Omnianomaly training curves on the WaDi dataset in Appendix 4.

6 Conclusion

In addition to issues with datasets and metrics discussed in previous works, we argue that problems in experimental design and reporting also contribute to unreproducible results in deep learning for time series anomaly detection. Through a review of influential works in this topic, we have identified several issues.

Eleven out of 18 examined papers suffer leakage from thresholding methods and the point-adjust algorithm. Seven papers are misusing datasets through undisclosed modifications. Leakage leads to overoptimistic evaluations of deep learning methods by optimizing for metrics. Misusing datasets casts doubt on the experimental results no matter the quality of the underlying dataset or metric and confuses comparison. Underspecification of the baseline comparison methods as seen in eleven papers makes it difficult for readers to assess the details and quality of the experimental setup.

In our empirical investigation of common baseline methods, we show that many published baseline results are not reproducible. Despite intentionally choosing suboptimal parameters for the simpler models, we see higher performance than many publications report. We often cannot reject similar performance between optimized deep learning models with trusted implementations, unoptimzed deep learning models, and unoptimized classical models. On the UCR dataset we even see classical methods significantly outperforming deep models under both AUC-ROC and AUC-PRC.

The question we want to ask is why? Due to leakage, modified datasets, and underspecification we lack the information to clearly explain why we do not see dramatic performance differences between traditional and deep learning models as often presented in previous works. Leakage and modified datasets distort the ability to asses true performance. Underspecification hides whether the difference may lie with hyperparameters, data pipeline, or some other detail of implementation. When comparison against baseline methods is the benchmark for measuring progress in the field, we argue that the reproducibility of baseline method results is just as critical as the main results. It is imperative that the baseline results are reasonable and trustworthy.

Directly resolving such fundamental issues is not a straightforward task. Previous work in the time series anomaly detection field have proposed higher quality datasets, such as

UCR and TSB-UAD (Wu & Keogh, 2022; Liu et al., 2024), and evaluation metrics, such as VUS (Boniol et al., 2025), to address reproducibility concerns. However, problematic elements remain in usage. Instead of proposing another competing standard, we instead advocate for increased transparency and clear disclosure of the entire experimental setup. With more clear reporting in the text and by releasing the full code, including the full data preprocessing pipeline and baseline experiments, we allow other authors and practitioners to judge for themselves the feasibility of the experimental setup for their own usage.

Possible directions for future work include further investigation of detection performance using metrics which are more robust to noisy labels or which take detection timing into account and deeper investigation into the surprising performance of classical methods and the effects of different types of anomalies. Additionally, a basic framework for comparison between state-of-the-art deep learning and classical methods should be developed to bolster collaboration and facilitate more robust measures of progress in the time series anomaly detection field.

Splitting the UCR Dataset

The UCR dataset consists of 250 entities collection from a wide variety of sensors (Wu & Keogh, 2022). We have categorized these entities into ten groups of similar sensor types and tasks and one miscellaneous group consisting of unique tasks. After filtering for artificially perturbed entities as described in Sect. 4, the remaining entities and groups are shown in Table 7.

Of these ten we will display the critical difference diagrams of the ECG, blood pressure, EPG, and band deformation groups in Figs. 4 and 5.

Table 7 The groups of entities within the UCR dataset after filtering

Group Type	Entities
ECG	109, 110, 111, 112, 119, 120, 121, 122, 123, 124, 125, 126, 163, 164, 165, 166, 178, 179, 180, 182, 183, 192, 193, 194, 195, 196, 226, 227, 228, 229, 230, 231, 232, 233, 234, 235, 236, 237
Blood pressure	132, 133, 134, 135, 136, 137, 138, 139, 140, 141, 142, 143, 144, 197, 198, 199, 200, 242, 243, 244, 245, 246, 247
EPG	145, 146, 147, 148, 149, 150, 173, 174, 175, 176, 177
Band Deformation	184, 185, 186, 187, 188, 189, 190, 191
Gait Force	167, 168, 169, 170, 171, 172, 181
Temperature	113, 114, 115, 116, 117, 118
MSL	156, 157, 158, 159, 160
Gait Distance	127, 128, 129, 130, 131
Power Demand	152, 153, 154, 155, 210, 211, 212
Accelerometer	151, 161, 162
Misc	201, 202, 203, 204, 205, 206, 207, 208, 209, 248, 249, 250

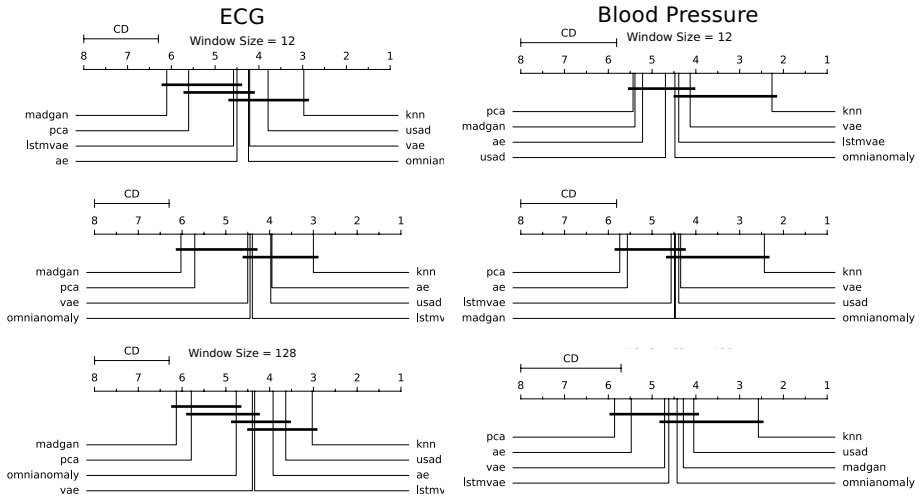


Fig. 4 Critical difference diagrams for AUC-ROC ($\alpha = 0.05$) on the ECG and blood pressure groups in the UCR dataset at various window lengths

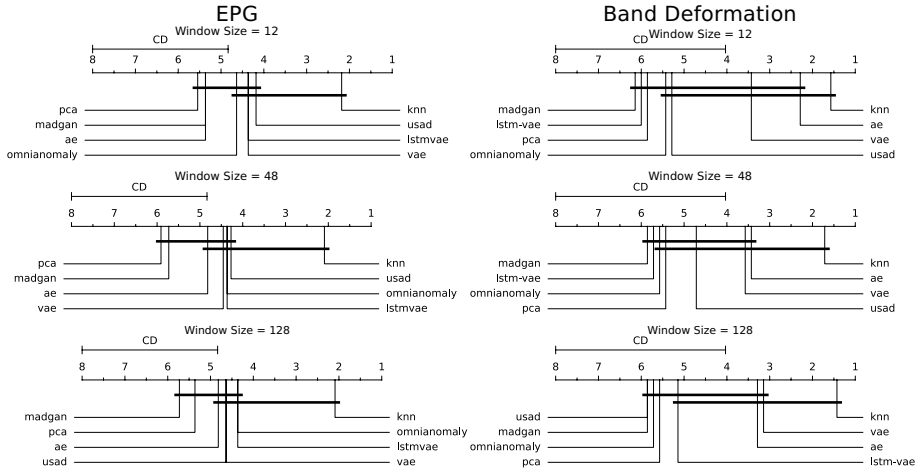


Fig. 5 Critical difference diagrams for AUC-ROC ($\alpha = 0.05$) on the EPG and band deformation groups in the UCR dataset at various window lengths

Hyperparameter Investigation

Hyperparameters are known to be a critical factor affecting model performance. Despite this, reporting of baseline hyperparameters and subsequent tuning is often overlooked. To investigate the possible effects of hyperparameter choice, we conduct a limited investigation on the SWaT and SMD datasets in Tables 8 and 9. We present results on models which

Table 8 Results of hyperparameter investigation on the SwAT dataset using the average AUC-ROC score over five runs

Hidden dim	64	128	256
AE	0.825 ± 0.004	0.826 ± 0.003	0.823 ± 0.005
VAE	0.820 ± 0.0	0.820 ± 0.0	0.820 ± 0.0
LSTM-VAE	0.820 ± 0.0	0.820 ± 0.0	0.820 ± 0.0
Hidden dim	250	500	750
Omnianomaly	0.820 ± 0.0	0.656 ± 0.265	0.821 ± 0.001
k	5	10	20
k-nn	0.824	0.824	0.825
solver	mle	default	
pca	0.824	0.812	

The middle column is the parameters used in the empirical investigation

Table 9 Results of hyperparameter investigation on the SMD dataset using the average AUC-ROC score over all entities

Hidden dim	32	64	128
AE	0.801 ± 0.103	0.796 ± 0.104	0.795 ± 0.103
VAE	0.771 ± 0.129	0.771 ± 0.129	0.771 ± 0.128
LSTM-VAE	0.771 ± 0.129	0.769 ± 0.129	0.771 ± 0.129
Hidden dim	50	100	200
MADGAN	0.669 ± 0.205	0.641 ± 0.178	0.605 ± 0.228
k	5	10	20
k-nn	0.790	0.824	0.825
solver	mle	default	
pca	0.758	0.812	

The middle column is the parameters used in the empirical investigation

did not have hyperparameters for the specified dataset in their original papers. In the case of PCA, we use scikit-learns mle solver parameter which automatically chooses the number of components.

Interestingly, Omnianomaly see's an improvement using smaller and larger hyperparameters, possibly indicating some instability in the implementation when using the original hyperparameters given by Su et al. (2019).

Results Using Area Under Precision Recall Curve

Table 10 presents AUC-PRC results on the SWaT dataset and Table 11 presents results on WaDi. Similarly, Fig. 6 shows the critical difference diagrams of the SMAP and MSL datasets and Fig. 7 the SMD and UCR datasets. These results were correspond to the same experiments from which the AUC-ROC scores in Sect. 5 were recorded. In general, the results using AUC-PRC resemble that of AUC-ROC. Performance is between categories of methods are similar with no clear advantage to complex tuned methods or disadvantage to traditional methods.

Additionally, we include the critical difference diagrams of the AUC-PRC for the ECG, blood pressure, EPG, and band deformation groups as described in Appendix 1 in Figs. 8 and 9.

Table 10 Comparison of mean and standard deviation of the AUC-PRC score on the SwaT dataset at window size 12, 48, and 128 across five runs

Model	SwaT-12	SwaT-48	SwaT-128
k-NN	0.727	0.727	0.727
PCA	0.719	0.716	0.716
AE	0.726 ± 0.002	0.726 ± 0.003	0.722 ± 0.006
VAE	**0.731 ± 0.0**	**0.731 ± 0.0**	0.731 ± 0.0
LSTM-VAE	**0.731 ± 0.0**	**0.731 ± 0.0**	**0.731 ± 0.0**
Omnianomaly	0.729 ± 0.004	0.567 ± 0.268	0.605 ± 0.282
MADGAN	0.717 ± 0.013	0.726 ± 0.014	0.715 ± 0.019
USAD	0.724 ± 0.010	0.721 ± 0.003	0.723 ± 0.006

Bold indicates highest score

Table 11 Comparison of mean and standard deviation of the AUC-PRC score on the WaDi dataset at window size 12, 48, and 128 across five runs

Model	WaDi-12	WaDi-48	WaDi-128
k-NN	0.204	0.204	0.204
PCA	0.056	0.056	0.056
AE	0.204 ± 0.001	0.205 ± 0.001	0.205 ± 0.0
VAE	**0.206 ± 0.0**	**0.206 ± 0.0**	**0.207 ± 0.0**
LSTM-VAE	**0.206 ± 0.0**	**0.206 ± 0.0**	0.206 ± 0.0
Omnianomaly	**0.206 ± 0.0**	**0.206 ± 0.0**	0.206 ± 0.0
MADGAN	0.092 ± 0.035	0.072 ± 0.032	0.147 ± 0.085
USAD	0.203 ± 0.0	0.202 ± 0.003	0.206 ± 0.001

Bold indicates highest score

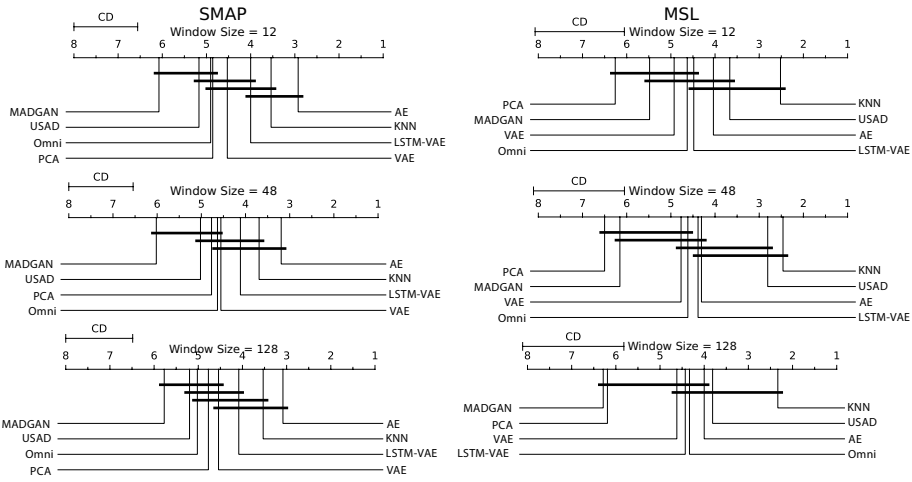


Fig. 6 Critical difference diagrams for AUC-PRC scores ($\alpha = 0.05$) on the SMAP and MSL datasets at various window lengths

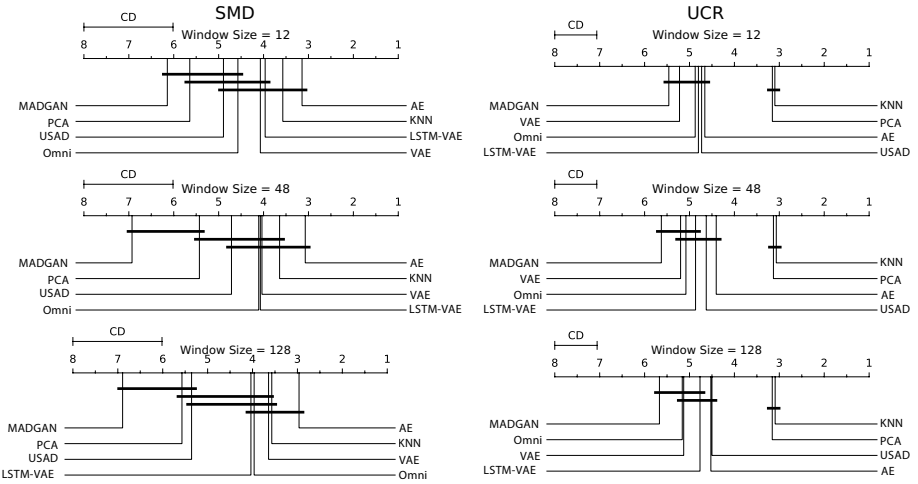


Fig. 7 Critical difference diagrams for AUC-PRC scores ($\alpha = 0.05$) on the SMD and UCR datasets at various window lengths

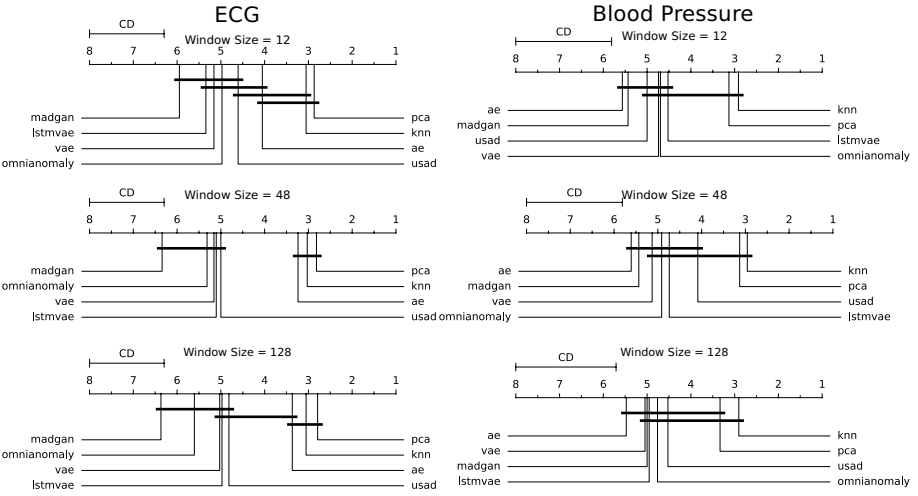


Fig. 8 Critical difference diagrams for AUC-PRC ($\alpha = 0.05$) on the ECG and blood pressure groups in the UCR dataset at various window lengths

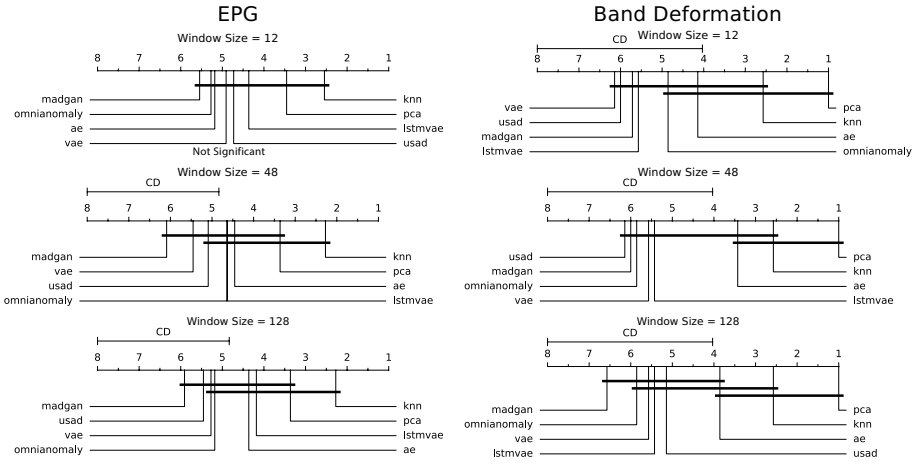


Fig. 9 Critical difference diagrams for AUC-PRC ($\alpha = 0.05$) on the EPG and band deformation groups in the UCR dataset at various window lengths

Deeper Investigation of Model Training on WaDi

The similar performance of VAE, LSTM-VAe, and Omnianomaly may indicate instability in training. As we can see from the loss curves in Fig. 10, the models are training but VAE

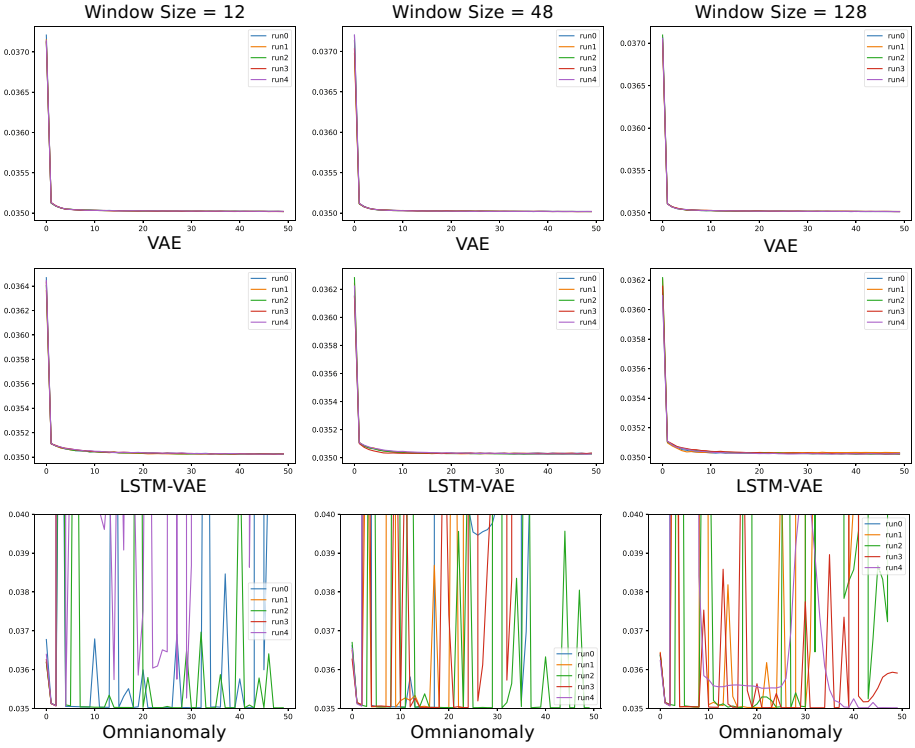


Fig. 10 Model losses during training

and LSTM-VAE converge extremely quickly. Omnianomaly is unstable and struggles to converge at all, resulting in training being terminated early. As seen in Appendix 2, there may be some incompatibility between the implementation by Liu et al. (2024) and original hyperparameters given by Su et al. (2019).

However, this result expected. Poor overall detection performance on WaDi has been reported previously (Garg et al., 2022). This dataset is challenging and it is unsurprising that simple VAE and LSTM-VAE models with non-optimized hyperparameters perform poorly. What is more surprising is that more advanced models using original or trusted implementations with hyperparameters optimized for this dataset are not performing better than purposefully suboptimal VAE and LSTM-VAE implementations.

What we wish to highlight is the irreproducibility of baseline model results, not just of main methods. Due to the issues described in Sect. 3, the reason for the difference between ours, Garg et al. (2022), and the very good detection performance on the WaDi dataset seen in other works (Li et al., 2019; Tuli et al., 2022) using the same methods is not explainable.

Acknowledgements This work is supported by the Wallenberg AI, Autonomous Systems and Software Program (WASP) funded by the Knut and Alice Wallenberg Foundation. The computations were enabled by resources provided by the National Academic Infrastructure for Supercomputing in Sweden (NAISS), partially funded by the Swedish Research Council through grant agreement no. 2022-06725.

Author Contributions N.X carried out the experiments and wrote the main manuscript text. Both authors reviewed the manuscript. H.B supervised the project.

Data Availability The SMaP and MSL datasets are available at https://www.kaggle.com/datasets/patrickfleith/nasa-anomaly-detection-dataset-smap-msl. The SMD dataset is available at https://www.kaggle.com/datasets/mgusat/smd-onmiad. The UCR anomaly detection dataset is available at https://wu.renjie.im/research/anomaly-benchmarks-are-flawed/#ucr-time-series-anomaly-archive. The SWaT and WaDi datasets are available upon request from iTrust labs https://itrust.sutd.edu.sg/itrust-labs_datasets/dataset_info/.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

References

Abdulaal, A., Liu, Z., & Lancewicki, T. (2021) Practical approach to asynchronous multivariate time series anomaly detection and localization. In: *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*. KDD '21, pp. 2485–2494. Association for Computing Machinery. https://doi.org/10.1145/3447548.3467174

Ahmed, C. M., Palleti, V. R., & Mathur, A. P. (2017). Wadi: a water distribution testbed for research in the design of secure cyber physical systems. *Proceedings of the 3rd International Workshop on Cyber-Physical Systems for Smart Water Networks. CySWATER '17* (pp. 25–28). New York, NY, USA: Association for Computing Machinery. https://doi.org/10.1145/3055366.3055375

An, J., & Cho, S. (2015). Variational autoencoder based anomaly detection using reconstruction probability. Special Lecture on IE, 2(1),

Audibert, J., Michiardi, P., Guyard, F., Marti, S. & Zuluaga, M. A. (2020). Usad: Unsupervised anomaly detection on multivariate time series. In: *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. KDD '20, pp. 3395–3404. Association for Computing Machinery, New York, NY, USA. https://doi.org/10.1145/3394486.3403392

Audibert, J., Michiardi, P., Guyard, F., Marti, S., & Zuluaga, M. A. (2022). Do deep neural networks contribute to multivariate time series anomaly detection? *Pattern Recognition, 132*, Article 108945. https://doi.org/10.1016/j.patcog.2022.108945

Benkabou, S.-E., Benabdeslem, K., Mansouri, D. E. K., Chaib, S., Mesmoudi, A., & Hadjali, A. (2025). A temporal dependency preserving approach for anomaly detection on multivariate time series. *World Wide Web, 28*(3), 37. https://doi.org/10.1007/s11280-025-01346-y

Boniol, P., Krishna, A. K., Bruel, M., Liu, Q., Huang, M., Palpanas, T., Tsay, R. S., Elmore, A., & Franklin, M. J. (2025). VUS: Effective and efficient accuracy measures for time-series anomaly detection. *The VLDB Journal*. https://doi.org/10.1007/s00778-025-00907-x

Chaovalitwongse, W. A., Fan, Y.-J., & Sachdeo, R. C. (2007). On the time series k-nearest neighbor classification of abnormal brain activity. *IEEE Transactions on Systems, Man, and Cybernetics - Part A: Systems and Humans, 37*(6), 1005–1016. https://doi.org/10.1109/TSMCA.2007.897589

Chen, Z., Chen, D., Zhang, X., Yuan, Z., & Cheng, X. (2021). Learning graph structures with transformer for multivariate time series anomaly detection in IoT. *IEEE Internet of Things Journal*. https://doi.org/10.1109/JIOT.2021.3100509

Demšar, J. (2006). Statistical comparisons of classifiers over multiple data sets. *J. Mach. Learn. Res., 7*, 1–30.

Deng, A., & Hooi, B. (2021). Graph neural network-based anomaly detection in multivariate time series. *Proceedings of the AAAI Conference on Artificial Intelligence* (Vol. 35, pp. 4027–4035)

Garg, A., Zhang, W., Samaran, J., Savitha, R., & Foo, C.-S. (2022). An evaluation of anomaly detection and diagnosis in multivariate time series. *IEEE Transactions on Neural Networks and Learning Systems, 33*(6), 2508–2517. https://doi.org/10.1109/TNNLS.2021.3105827

Goh, J., Adepu, S., Junejo, K. N., & Mathur, A. (2017). A dataset to support research in the design of secure water treatment systems. In G. Havarneanu, R. Setola, H. Nassopoulos, & S. Wolthusen (Eds.), *Critical Information Infrastructures Security* (pp. 88–99). Springer.

Hamilton, J. D. (1994). *Time Series Analysis*. Princeton University Press.

He, Y., & Zhao, J. (2019). Temporal convolutional networks for anomaly detection in time series. *Journal of Physics: Conference Series, 1213*(4), Article Article 042050. https://doi.org/10.1088/1742-6596/1213/4/042050

Herbold, S. (2020). Autorank: A python package for automated ranking of classifiers. *Journal of Open Source Software, 5*(48), Article 2173. https://doi.org/10.21105/joss.02173

Hundman, K., Constantinou, V., Laporte, C., Colwell, I., & Soderstrom, T. (2018) Detecting spacecraft anomalies using lstms and nonparametric dynamic thresholding. In: Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining. KDD '18, pp. 387–395. Association for Computing Machinery, New York, NY, USA . https://doi.org/10.1145/3219819.3219845

Kapoor, S., & Narayanan, A. (2023). Leakage and the reproducibility crisis in machine-learning-based science. *Patterns*. https://doi.org/10.1016/j.patter.2023.100804. Publisher: Elsevier.

Lakhina, A., Crovella, M., & Diot, C. (2004). *Characterization of network-wide anomalies in traffic flows* (pp. 201–206). Association for Computing Machinery. https://doi.org/10.1145/1028788.1028813 IMC '04.

Li, D., Chen, D., Jin, B., Shi, L., Goh, J., & Ng, S.-K. (2019). Mad-gan: Multivariate anomaly detection for time series data with generative adversarial networks. In I. V. Tetko, V. Kůrková, P. Karpov, & F. Theis (Eds.), *Artificial Neural Networks and Machine Learning - ICANN 2019: Text and Time Series* (pp. 703–716). Springer.

Li, Z., Zhao, Y., Han, J., Su, Y., Jiao, R., Wen, X., & Pei, D. (2021). Multivariate time series anomaly detection and interpretation using hierarchical inter-metric and temporal embedding. In: *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*, pp. 3220–3230

Liu, Q., & Paparrizos, J. (2024). The elephant in the room: Towards a reliable time-series anomaly detection benchmark. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, & C. Zhang (Eds.), *Advances in Neural Information Processing Systems* (Vol. vol. 37, pp. 108231–108261). Curran Associates, Inc. https://doi.org/10.52202/079017-3437

Ma, J., & Perkins, S. (2003). Time-series novelty detection using one-class support vector machines. *Proceedings of the International Joint Conference on Neural Networks* (Vol. 3, pp. 1741–17453). https://doi.org/10.1109/IJCNN.2003.1223670

Malhotra, P., Vig, L., Shroff, G. M., & Agarwal, P. (2015). Long short term memory networks for anomaly detection in time series. The European Symposium on Artificial Neural Networks

Mathur, A. P., & Tippenhauer, N. O. (2016). Swat: a water treatment testbed for research and training on ics security. *2016 International Workshop on Cyber-physical Systems for Smart Water Networks (CySWater)* (pp. 31–36). https://doi.org/10.1109/CySWater.2016.7469060

Munir, M., Siddiqui, S. A., Dengel, A., & Ahmed, S. (2019). Deepant: A deep learning approach for unsupervised anomaly detection in time series. *IEEE Access, 7*, 1991–2005. https://doi.org/10.1109/ACCESS.2018.2886457

Nakamura, T., Imamura, M., Mercer, R., & Keogh, E. (2020). Merlin: Parameter-free discovery of arbitrary length anomalies in massive time series archives. *2020 IEEE International Conference on Data Mining (ICDM)* (pp. 1190–1195). https://doi.org/10.1109/ICDM50108.2020.00147

Nanduri, A., & Sherry, L. (2016). Anomaly detection in aircraft data using recurrent neural networks (rnn). *2016 Integrated Communications Navigation and Surveillance (ICNS)* (pp. 5–21528). https://doi.org/10.1109/ICNSURV.2016.7486356

Park, D., Hoshi, Y., & Kemp, C. C. (2018). A multimodal anomaly detector for robot-assisted feeding using an lstm-based variational autoencoder. *IEEE Robotics and Automation Letters, 3*(3), 1544–1551. https://doi.org/10.1109/LRA.2018.2801475

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Kopf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., & Chintala, S. (2019) Pytorch: An imperative style, high-performance deep learning library, 8024–8035

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.

Ramaswamy, S., Rastogi, R., & Shim, K. (2000). Efficient algorithms for mining outliers from large data sets. *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data. SIGMOD '00* (pp. 427–438). New York, NY, USA: Association for Computing Machinery. https://doi.org/10.1145/342009.335437

Sakurada, M., & Yairi, T. (2014). Anomaly detection using autoencoders with nonlinear dimensionality reduction. *Proceedings of the MLSDA 2014 2nd Workshop on Machine Learning for Sensory Data Analysis. MLSDA'14* (pp. 4–11). New York, NY, USA: Association for Computing Machinery. https://doi.org/10.1145/2689746.2689747

Schmidl, S., Wenig, P., & Papenbrock, T. (2022). Anomaly detection in time series: A comprehensive evaluation. *Proceedings of the VLDB Endowment, 15*(9), 1779–1797. https://doi.org/10.14778/3538598.3538602

Sculley, D., Snoek, J., Wiltschko, A. B., & Rahimi, A. (2018). Winner's curse? on pace, progress, and empirical rigor. International Conference on Learning Representations

Siwon, K., Choi, K., Choi, H.-S., Lee, B., & Yoon, S. (2022). Towards a rigorous evaluation of time-series anomaly detection. *Proceedings of the AAAI Conference on Artificial Intelligence, 36*, 7194–7201. https://doi.org/10.1609/aaai.v36i7.20680

Su, Y., Zhao, Y., Niu, C., Liu, R., Sun, W., & Pei, D. (2019) Robust anomaly detection for multivariate time series through stochastic recurrent neural network. In: Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining. KDD '19, pp. 2828–2837. Association for Computing Machinery, New York, NY, USA. https://doi.org/10.1145/3292500.3330672

Taylor, S. J., & Letham, B. (2018). Forecasting at scale. *The American Statistician, 72*(1), 37–45. https://doi.org/10.1080/00031305.2017.1380080

Tuli, S., Casale, G., & Jennings, N. R. (2022). Tranad: Deep transformer networks for anomaly detection in multivariate time series data. *Proceedings of the VLDB Endowment, 15*(6), 1201–1214. https://doi.org/10.14778/3514061.3514067

Wagner, D., Michels, T., Schulz, F. C. F., Nair, A., Rudolph, M., & Kloft, M. (2023). TimeseAD: Benchmarking deep multivariate time-series anomaly detection. Transactions on Machine Learning Research

Wu, H., Hu, T., Liu, Y., Zhou, H., Wang, J., & Long, M. (2023). Timesnet: Temporal 2d-variation modeling for general time series analysis International Conference on Learning Representations.

Wu, R., & Keogh, E. J. (2022). Current time series anomaly detection benchmarks are flawed and are creating the illusion of progress (extended abstract). *2022 IEEE 38th International Conference on Data Engineering (ICDE)* (pp. 1479–1480). https://doi.org/10.1109/ICDE53745.2022.00116

Xu, J., Wu, H., Wang, J., & Long, M. (2022). Anomaly transformer: Time series anomaly detection with association discrepancy. International Conference on Learning Representations

Yeh, C.-C.M., Zhu, Y., Ulanova, L., Begum, N., Ding, Y., Dau, H. A., Silva, D. F., Mueen, A., & Keogh, E. (2016). Matrix profile i: All pairs similarity joins for time series: A unifying view that includes motifs, discords and shapelets. *IEEE 16th International Conference on Data Mining (ICDM)* (pp. 1317–1322). https://doi.org/10.1109/ICDM.2016.0179

Zamanzadeh Darban, Z., Webb, G. I., Pan, S., Aggarwal, C., & Salehi, M. (2024). Deep learning for time series anomaly detection: A survey. *ACM Computing Surveys*. https://doi.org/10.1145/3691338

Zhou, B., Liu, S., Hooi, B., Cheng, X., & Ye, J. (2019). Beatgan: Anomalous rhythm detection using adversarially generated time series. International Joint Conference on Artificial Intelligence

Zong, B., Song, Q., Min, M. R., Cheng, W., Lumezanu, C., Cho, D., & Chen, H. (2018). Deep autoencoding gaussian mixture model for unsupervised anomaly detection. International Conference on Learning Representations

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.